



Universidade Federal do Amazonas
Instituto de Ciências Exatas e Tecnologia
Bacharelado em Engenharia de Software



Arthur Djan Pinheiro Pontes,
Pâmilla Nunes Santos,
Heloise Heloá Serrão Viana,
Jeziel Monteiro Pessoa,
Sérgio Augusto Simões da Silva

Porto Certo: Reserva de Passagens Fluviais

Projeto de Software

Itacoatiara - Am
2026

SUMÁRIO

Sumário	1
1 Introdução	2
2 Escopo do Projeto	2
3 Organização do Projeto	3
4 Equipe e infraestrutura	4
4.1 Equipamentos e Hardware	4
4.2 Ferramentas de Software	4
4.3 Ambientes de Desenvolvimento e Servidores	5
4.4 Planejamento de Alocação de Pessoal	6
5 Requisitos	6
5.1 Histórias de Usuário	6
6 Modelo Conceitual	8
6.1 Entidades	8
6.2 Relacionamentos	9
7 Modelo Lógico	10
8 Modelo Físico	14
8.1 Ambiente Tecnológico e Configuração do SGBD	15
8.2 Script de Implementação Física (DDL)	15
8.3 Dicionário de Dados e Tipagem Detalhada	15
8.4 Análise de Decisões Técnicas	18
8.5 Hierarquia e Integridade Referencial	18
8.6 Implementação de Regras de Negócio e Restrições Físicas	18

1 INTRODUÇÃO

O presente documento tem como objetivo apresentar o desenvolvimento de um sistema digital para a comercialização de passagens fluviais, voltado à unificação de informações relacionadas ao transporte hidroviário, permitindo a compra e reserva de passagens online, bem como o gerenciamento de serviços de transporte. O sistema destina-se a passageiros e operadores de transporte fluvial, oferecendo funcionalidades específicas para cada perfil de utilização.

O projeto surge diante da necessidade de centralizar informações sobre viagens fluviais, como valores de passagens, datas de embarque e desembarque locais de partida, reduzindo a dependência de informações obtidas por terceiros.

Dessa forma, a proposta visa proporcionar maior facilidade de acesso às informações, além de automatizar processos de compra, reserva e gerenciamento comercial do transporte fluvial em um único ambiente digital. móvel

O sistema será desenvolvido como um aplicativo , permitindo que passageiros realizem buscas por rotas fluviais, reservas e compras de passagens, acompanhamento geográfico de embarcações e consulta ao histórico de viagens. Para os proprietários e operadores, o sistema disponibilizará funcionalidades de cadastro e gestão de frotas, publicação de viagens, comunicação com passageiros e acompanhamento financeiro das operações.

No contexto deste documento, alguns termos técnicos são utilizados, tais como MVP (Minimum Viable Product), ETA (Estimated Time of Arrival) e Dashboard de Faturamento, relacionados às funcionalidades e à estrutura do sistema proposto.

Por fim, este documento poderá sofrer atualizações ao longo do ciclo de vida do projeto, acompanhando as etapas de ideação, levantamento de requisitos, projeto e arquitetura, elaboração de casos de teste e desenvolvimento do MVP, conforme a metodologia ágil Scrum adotada pela equipe.

2 ESCOPO DO PROJETO

O projeto consiste no desenvolvimento de um aplicativo móvel voltado à comercialização de passagens fluviais, com o objetivo de unificar informações relacionadas ao transporte hidroviário, permitindo o cadastro de passageiros, a busca de rotas, a consulta de viagens disponíveis e a realização de reservas de passagens online.

O sistema busca solucionar a dificuldade de acesso a informações sobre preços,

datas, horários e locais de embarque e desembarque, centralizando esses dados em uma única plataforma digital. Nesta versão simplificada do projeto, o sistema será direcionado principalmente aos passageiros, permitindo que eles se cadastrem, pesquisem viagens informando origem, destino e data, visualizem informações da embarcação, consultem o valor base da passagem e realizem reservas.

Para fins deste MVP, o modelo de dados será limitado às entidades essenciais para o funcionamento da reserva de passagens, contemplando Passageiro, Reserva, Viagem, Rota, Embarcação e Tipo de Embarcação.

Como delimitação do projeto, não serão implementados nesta versão módulos de pagamento, rastreamento geográfico em tempo real, gerenciamento financeiro, comunicação com passageiros, autenticação avançada, gestão completa de operadores/proprietários, transporte de cargas, mercadorias ou veículos pesados, nem algoritmos de Inteligência Artificial para recomendação ou precificação dinâmica de passagens.

3 ORGANIZAÇÃO DO PROJETO

A organização do projeto é composta por uma equipe responsável pelo desenvolvimento do sistema de comercialização de passagens fluviais, atuando de forma colaborativa nas etapas de levantamento de requisitos, modelagem, arquitetura, desenvolvimento, testes e validação do sistema. A estrutura organizacional foi definida de acordo com os papéis e responsabilidades atribuídos a cada integrante da equipe, visando garantir uma divisão clara das atividades e um fluxo eficiente de comunicação e tomada de decisões.

A hierarquia organizacional do projeto possui o Product Owner como principal responsável pela definição e validação das decisões relacionadas aos requisitos e prioridades do sistema. As decisões técnicas relacionadas à arquitetura e modelagem são conduzidas pelo Software Architect em conjunto com a equipe de desenvolvimento.

A comunicação entre os integrantes ocorre de forma colaborativa durante as sprints definidas pela metodologia Scrum, permitindo acompanhamento contínuo das atividades, alinhamento das entregas e resolução de problemas ao longo do desenvolvimento do projeto. O fluxo de decisões ocorre de maneira integrada entre os membros da equipe, considerando os requisitos do sistema, as necessidades dos utilizadores e os objetivos do projeto.

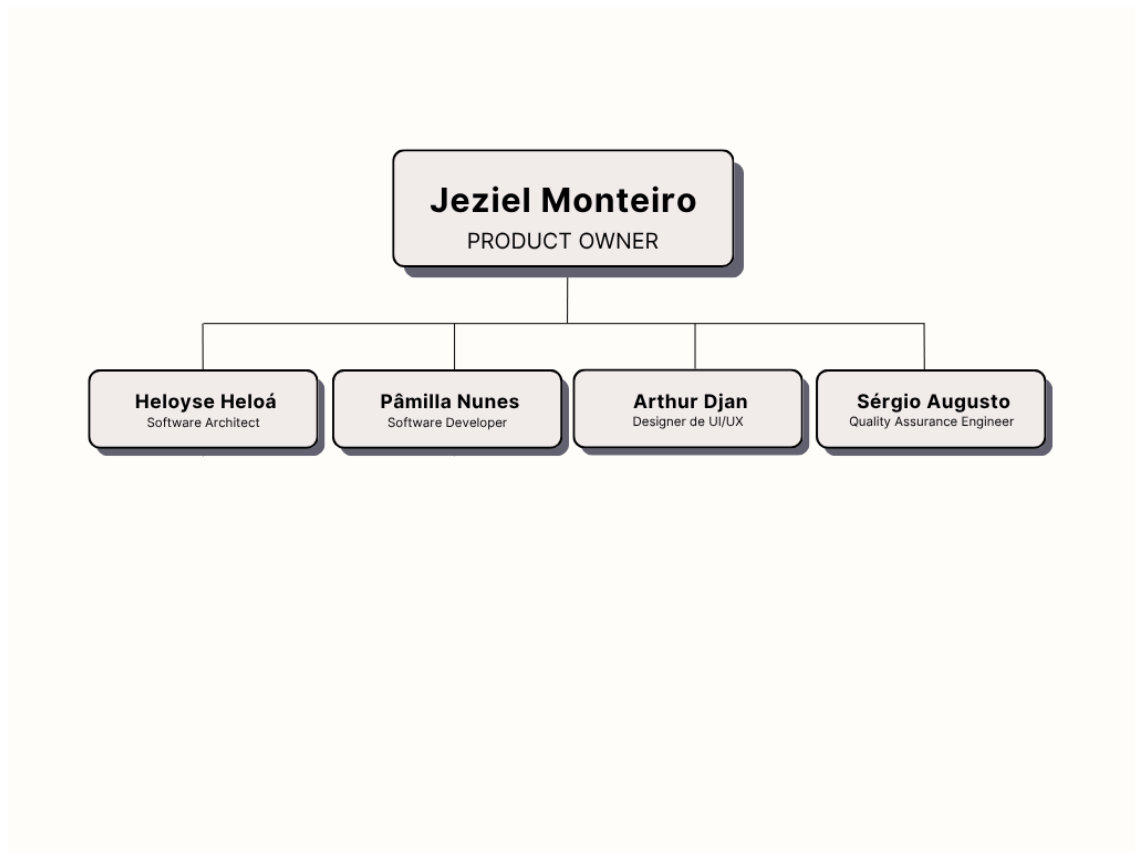


Figura 1 – FLuxograma da equipe.

4 EQUIPE E INFRAESTRUTURA

Para garantir a eficiência durante o ciclo de vida do desenvolvimento de software, o projeto requer os seguintes recursos de infraestrutura corporativa e tecnológica:

4.1 Equipamentos e Hardware

- **Estações de trabalho** (computadores desktop ou notebooks) equipadas com processadores de múltiplos núcleos, memória RAM mínima de 16 GB e armazenamento SSD, essenciais para suportar a execução simultânea de IDEs, contêineres e bancos de dados locais.

4.2 Ferramentas de Software

- **Design e Prototipação:** Figma, utilizado para a concepção do layout, diagramação das interfaces e fluxos de navegação do passageiro.

Integrante	Papel	Responsabilidades
Jeziel Monteiro	Product Owner (PO)	Responsável pela definição dos requisitos, priorização das funcionalidades e acompanhamento geral do projeto
Heloise Heloá	Software Architect	Responsável pela arquitetura, modelagem e organização estrutural do sistema
Pâmilla Nunes	Software Developer	Responsável pela implementação e desenvolvimento das funcionalidades do sistema
Sérgio Augusto	Quality Assurance (QA) Engineer	Responsável pelos testes, validação e garantia da qualidade do sistema
Arthur Djan	Designer UI/UX	Responsável pela elaboração da interface e experiência do usuário

Tabela 1 – Integrantes, papéis e responsabilidades da equipe

- **Controle de Versão:** Git, operando preferencialmente via interface de linha de comando (terminais nativos), integrado a um repositório remoto (como GitHub) para o gerenciamento do código-fonte e documentação em formato Markdown.
- **Ambientes de Desenvolvimento Integrado (IDEs):** Ambientes robustos adaptados para as tecnologias escolhidas (ex.: IntelliJ IDEA para o desenvolvimento *Back-end* e Visual Studio Code para o *Front-end*).
- **Sistemas de Gerenciamento de Banco de Dados (SGBD):** SGBD relacional (como PostgreSQL ou MySQL) para o armazenamento estruturado das informações de reservas, rotas e clientes.

4.3 Ambientes de Desenvolvimento e Servidores

- **Ambientes Locais:** Máquinas operando sob sistemas operacionais baseados em Linux (como Ubuntu) e Windows, garantindo a compatibilidade multiplataforma das ferramentas.
- **Servidores de Homologação/Produção:** Contratação de infraestrutura em nuvem (*Cloud Computing*, como AWS ou Azure) para a hospedagem do banco de dados em rede e para o *deploy* da API, garantindo a disponibilidade do serviço para testes externos.

4.4 Planejamento de Alocação de Pessoal

A distribuição de recursos humanos ocorrerá de maneira progressiva, alinhando-se às fases do ciclo de desenvolvimento do software, de forma a otimizar o tempo e garantir a qualidade técnica das entregas.

Fase 1: Levantamento de Requisitos e Modelagem

Alocação: Foco primário do Analista de Requisitos/DBA para a definição das regras de negócio e estruturação do banco de dados. O Desenvolvedor UI/UX atua em paralelo na prototipação das telas. O Desenvolvedor *Back-end* auxilia na validação da viabilidade técnica da arquitetura.

Fase 2: Estruturação de Banco de Dados e *Back-end*

Alocação: O DBA finaliza a criação das tabelas e relacionamentos. O Desenvolvedor *Back-end* assume a carga horária principal, iniciando o desenvolvimento das rotas da API e da lógica de persistência. O Desenvolvedor *Front-end* configura o ambiente de trabalho e inicia a codificação base do sistema.

Fase 3: Desenvolvimento *Front-end* e Integração

Alocação: O Desenvolvedor *Front-end* assume o papel de liderança técnica nesta fase, conectando as interfaces visuais à API. O Desenvolvedor *Back-end* fornece suporte contínuo para ajustes e correção de falhas de comunicação (*endpoints*).

Fase 4: Testes, Homologação e Implantação

Alocação: Dedicção integral de toda a equipe. O DBA monitora o desempenho das consultas; o Desenvolvedor *Back-end* avalia *logs* de erro do servidor; e o *Front-end* realiza testes de usabilidade e responsividade simulando o comportamento real do passageiro.

5 REQUISITOS

5.1 Histórias de Usuário

Persona: Passageiro

US01 — Como viajante, desejo me cadastrar no Porto Certo para que eu possa ter acesso às funcionalidades do aplicativo.

- Critérios de Aceitação

- O sistema deve permitir que o passageiro insira o CPF em uma caixa, contendo somente 11 dígitos;
- O sistema deve permitir que o passageiro insira o telefone em uma caixa, verificando o DDD e o número inserido;
- O sistema deve permitir que o passageiro insira o e-mail, validando o formato e verificando se não existe um e-mail igual já cadastrado;
- O sistema deve gerar um ID único para identificar o usuário cadastrado.

- **Regras de Negócio**

- Os campos CPF e E-mail devem ser chaves únicas no banco de dados, não permitindo duplicidade;
- O passageiro não pode ser cadastrado com dados em branco (campos obrigatórios: Nome, CPF e E-mail).

US02 — Como viajante, desejo buscar por viagens informando origem, destino e data, para encontrar opções de transporte disponíveis que atendam à minha necessidade.

- **Critérios de Aceitação**

- A tela de busca deve conter filtros obrigatórios para “Origem”, “Destino” e “Data”;
- O sistema deve exibir uma lista de resultados contendo o nome da embarcação, horário de saída, duração e valor base da passagem;
- As consultas de busca ao sistema devem ser respondidas e listadas na tela em até três segundos.

- **Regras de Negócio**

- Listar apenas viagens com vagas disponíveis e cuja data/hora de embarque seja no futuro (viagem ainda não realizada).

US03 — Como viajante, desejo realizar uma reserva de passagem para que eu possa viajar ao local em que eu queira ir.

- **Critérios de Aceitação**

- O sistema deve permitir que o passageiro selecione a viagem desejada e confirme a reserva;
- O sistema deve exibir o valor final da passagem antes da confirmação;
- O sistema deve registrar a data de criação da reserva e o status (ex.: Confirmada, Pendente);
- O sistema deve gerar um ID único da reserva;
- O sistema deve exibir uma mensagem de erro clara caso a viagem selecionada não tenha mais vagas no momento da confirmação.

• Regras de Negócio

- **Restrição de Capacidade:** O sistema deve verificar a lotação atual da viagem. A reserva não pode ser efetivada se o número de reservas daquela viagem atingir ou ultrapassar a capacidade máxima estipulada no cadastro da Embarcação associada à viagem;
- Um passageiro não pode reservar duas passagens para a mesma viagem (limitado a 1 reserva por ID de passageiro na mesma viagem).

6 MODELO CONCEITUAL

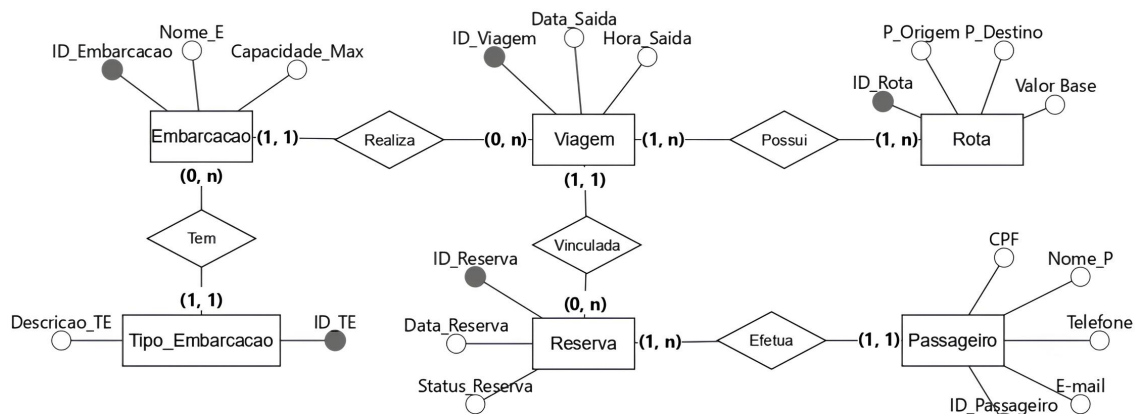


Figura 2 – Imagem do modelo conceitual do sistema.

6.1 Entidades

Embarcacao:

Armazena os dados físicos dos barcos disponíveis no sistema (possui ID, Nome e Capacidade Máxima).

Tipo_Embarcacao:

Serve para categorizar as embarcações, como “Lancha”, “Balsa” ou “Navio” (possui ID e Descrição).

Viagem:

Representa um evento específico no calendário, ou seja, uma saída programada (possui ID, Data e Hora de Saída).

Rota:

Define o trajeto padrão e o custo financeiro base do percurso (possui ID, Porto de Origem, Porto de Destino e Valor Base).

Reserva:

É o bilhete ou a passagem em si, que garante o lugar do cliente (possui ID, Data da Reserva e Status da Reserva).

Passageiro:

Guarda as informações pessoais dos clientes que utilizam o sistema (possui ID, CPF, Nome, Telefone e E-mail).

6.2 Relacionamentos

Tem (Embarcacao → Tipo_Embarcacao):

Classificar os barcos. Define que toda embarcação pertence a um tipo específico, e um tipo pode ser atribuído a várias embarcações.

Realiza (Embarcacao → Viagem):

Alocar barcos para as viagens. Define qual embarcação física fará uma determinada viagem agendada.

Possui (Viagem → Rota):

Estabelecer o trajeto da viagem. Define que uma viagem agendada precisa seguir uma rota pré-cadastrada no sistema.

Vinculada (Reserva → Viagem):

Garantir o assento. Conecta o bilhete comprado à viagem específica que vai acontecer em uma determinada data e hora.

Efetua (Passageiro → Reserva):

Registrar a posse da passagem. Associa a reserva ao cliente que a comprou, indicando de quem é aquele bilhete.

7 MODELO LÓGICO

Legenda: Campo = Chave Primária (PK) Campo = Chave Estrangeira (FK)

Tabela 2 – **Tipo_Embarcacao**

Campo	Tipo de Dado	Restrição/Chave	Regra de Integridade Referencial
<u>ID_TE</u>	Inteiro	PK (Chave Primária)	Identificador único da categoria da embarcação.
Descricao_TE	Caractere Variável (50)	Não Nulo	—

Tabela 3 – **Embarcacao**

Campo	Tipo de Dado	Restrição/Chave	Regra de Integridade Referencial
<u>ID_Embarcacao</u>	Inteiro	PK (Chave Primária)	Identificador único da embarcação.
Nome_E	Caractere Variável (100)	Não Nulo	—
Capacidade_Max	Inteiro	Não Nulo	—
<u>ID_TE</u>	Inteiro	FK (Chave Estrangeira)	Referencia a chave primária <u>ID_TE</u> na tabela Tipo_Embarcacao.

Tabela 4 – **Rota**

Campo	Tipo de Dado	Restrição/Chave	Regra de Integridade Referencial
<u>ID_Rota</u>	Inteiro	PK (Chave Primária)	Identificador único da rota.
P_Origem	Caractere Variável (100)	Não Nulo	—
P_Destino	Caractere Variável (100)	Não Nulo	—
Valor_Base	Decimal (10,2)	Não Nulo	—

Tabela 5 – Viagem

Campo	Tipo de Dado	Restrição/Chave	Regra de Integridade Referencial
<u>ID_Viagem</u>	Inteiro	PK (Chave Primária)	Identificador único da viagem.
Data_Saida	Data	Não Nulo	—
Hora_Saida	Hora	Não Nulo	—
<u>ID_Embarcacao</u>	Inteiro	FK (Chave Estrangeira)	Referencia a chave primária <u>ID_Embarcacao</u> na tabela Embarcacao.
<u>ID_Rota</u>	Inteiro	FK (Chave Estrangeira)	Referencia a chave primária <u>ID_Rota</u> na tabela Rota.

Tabela 6 – Passageiro

Campo	Tipo de Dado	Restrição/Chave	Regra de Integridade Referencial
<u>ID_Passageiro</u>	Inteiro	PK (Chave Primária)	Identificador único do passageiro.
CPF	Caractere Variável (14)	Único / Não Nulo	Restrição de unicidade para o documento do passageiro.
Nome_P	Caractere Variável (100)	Não Nulo	—
Telefone	Caractere Variável (20)	Opcional	—
E-mail	Caractere Variável (100)	Opcional	—

Tabela 7 – Reserva

Campo	Tipo de Dado	Restrição/Chave	Regra de Integridade Referencial
<u>ID_Reserva</u>	Inteiro	PK (Chave Primária)	Identificador único da reserva.
Data_Reserva	Data	Não Nulo	—
Status_Reserva	Caractere Variável (20)	Não Nulo	—
<u>ID_Viagem</u>	Inteiro	FK (Chave Estrangeira)	Referencia a chave primária <u>ID_Viagem</u> na tabela Viagem.
<u>ID_Passageiro</u>	Inteiro	FK (Chave Estrangeira)	Referencia a chave primária <u>ID_Passageiro</u> na tabela Passageiro.

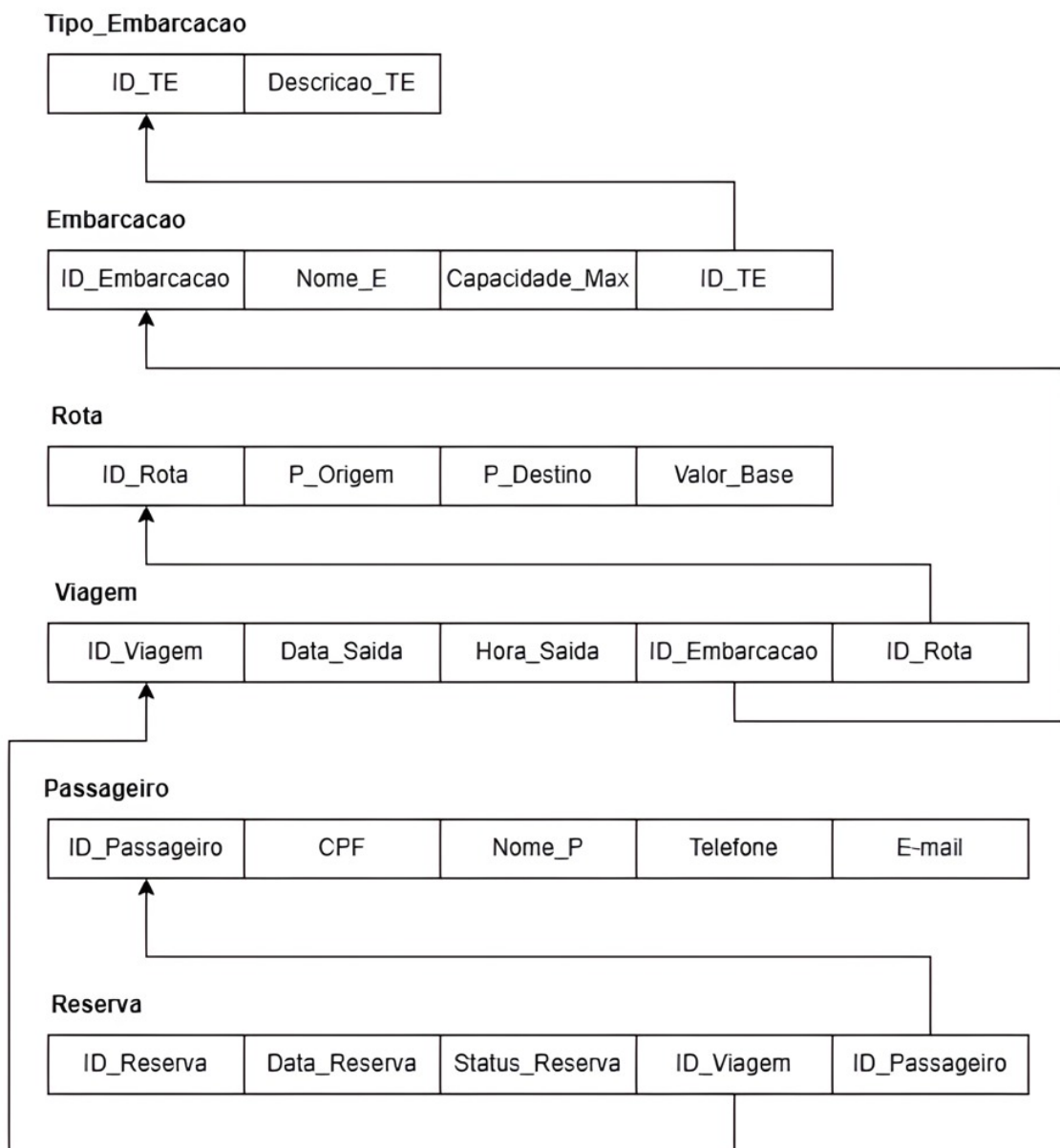


Figura 3 – Imagem do modelo lógico.

8 MODELO FÍSICO

```
portoCerto.sql

CREATE DATABASE IF NOT EXISTS PortoCerto;
USE PortoCerto;

CREATE TABLE Tipo_Embarcacao (
    ID_TE INT AUTO_INCREMENT PRIMARY KEY,
    Descricao_TE VARCHAR(100) NOT NULL
);

CREATE TABLE Rota (
    ID_Rota INT AUTO_INCREMENT PRIMARY KEY,
    P_Origem VARCHAR(100) NOT NULL,
    P_Destino VARCHAR(100) NOT NULL,
    Valor_Base DECIMAL(10, 2) NOT NULL
);

CREATE TABLE Passageiro (
    ID_Passageiro INT AUTO_INCREMENT PRIMARY KEY,
    CPF VARCHAR(14) UNIQUE NOT NULL,
    Nome_P VARCHAR(150) NOT NULL,
    Telefone VARCHAR(20),
    Email VARCHAR(100)
);

CREATE TABLE Embarcacao (
    ID_Embarcacao INT AUTO_INCREMENT PRIMARY KEY,
    Nome_E VARCHAR(100) NOT NULL,
    Capacidade_Max INT NOT NULL,
    ID_TE INT NOT NULL,
    CONSTRAINT fk_Embarcacao_Tipo FOREIGN KEY (ID_TE) REFERENCES Tipo_Embarcacao(ID_TE)
);

CREATE TABLE Viagem (
    ID_Viagem INT AUTO_INCREMENT PRIMARY KEY,
    Data_Saida DATE NOT NULL,
    Hora_Saida TIME NOT NULL,
    ID_Embarcacao INT NOT NULL,
    ID_Rota INT NOT NULL,
    CONSTRAINT fk_Viagem_Embarcacao FOREIGN KEY (ID_Embarcacao) REFERENCES Embarcacao(ID_Embarcacao),
    CONSTRAINT fk_Viagem_Rota FOREIGN KEY (ID_Rota) REFERENCES Rota(ID_Rota)
);

CREATE TABLE Reserva (
    ID_Reserva INT AUTO_INCREMENT PRIMARY KEY,
    Data_Reserva DATETIME DEFAULT CURRENT_TIMESTAMP,
    Status_Reserva VARCHAR(50) NOT NULL,
    ID_Viagem INT NOT NULL,
    ID_Passageiro INT NOT NULL,
    CONSTRAINT fk_Reserva_Viagem FOREIGN KEY (ID_Viagem) REFERENCES Viagem(ID_Viagem),
    CONSTRAINT fk_Reserva_Passageiro FOREIGN KEY (ID_Passageiro) REFERENCES Passageiro(ID_Passageiro)
);
```

Figura 4 – Imagem do modelo físico.

8.1 Ambiente Tecnológico e Configuração do SGBD

Para sustentar a integridade transacional do Porto Certo, a especificação técnica define o uso do **MySQL Community Server 8.0.46**. Esta versão foi selecionada por sua estabilidade comprovada e suporte a recursos modernos de automação, como Gatilhos (*Triggers*) e Procedimentos Armazenados. Como interface de gestão, utiliza-se o **MySQL Workbench**, permitindo o monitoramento visual e a administração eficiente da estrutura.

Para assegurar o suporte total a chaves estrangeiras e conformidade ACID (Atomicidade, Consistência, Isolamento e Durabilidade), o sistema utiliza obrigatoriamente o *Storage Engine* **InnoDB**. Esta escolha é fundamental para garantir que as restrições de integridade referencial sejam aplicadas pelo motor do banco.

As configurações de ambiente seguem os parâmetros abaixo:

- **Perfil do Servidor:** *Development Computer* (otimizado para o ciclo de construção).
- **Protocolo de Rede:** TCP/IP.
- **Porta de Comunicação:** 3306 (porta padrão MySQL).
- **Mecanismo de Armazenamento:** InnoDB (essencial para suporte a **CONSTRAINT** e **FOREIGN KEY**).

A configuração correta deste ambiente é o pré-requisito mandatório para a execução segura do *script* de definição de dados (DDL).

8.2 Script de Implementação Física (DDL)

A implementação do esquema Porto Certo respeita uma hierarquia de dependências. A execução deve seguir a ordem das tabelas independentes para as dependentes, garantindo que as referências de chaves estrangeiras encontrem seus alvos durante a criação. O script abaixo utiliza o tipo **DECIMAL(10,2)** para os campos de valor. A escolha é crítica para evitar a "imprecisão de ponto flutuante" inerente a tipos como **FLOAT** ou **DOUBLE**, garantindo que cálculos financeiros de bilhetes sejam exatos.

8.3 Dicionário de Dados e Tipagem Detalhada

A tipagem estrita é o que define a performance das consultas e a economia de armazenamento em escala. O uso de restrições **NOT NULL** em campos mandatórios

previne a existência de registros inconsistentes que poderiam quebrar a lógica de negócio na aplicação.

Tabela 8 – **Passageiro**

Campo	Tipo de Dado	Restrição	Descrição de Negócio
<u>ID_Passageiro</u>	INT	PK, AUTO_INCREMENT	Identificador único do passageiro.
CPF	VARCHAR(14)	UNIQUE, NOT NULL	CPF do passageiro (incluindo máscara de pontuação).
<u>Nome_P</u>	VARCHAR(150)	NOT NULL	Nome completo do titular da reserva.
Telefone	VARCHAR(20)	—	Contato para notificações de viagem.
Email	VARCHAR(100)	—	Canal de envio do bilhete eletrônico.

Tabela 9 – **Embarcação**

Campo	Tipo de Dado	Restrição	Descrição de Negócio
<u>ID_Embarcacao</u>	INT	PK, AUTO_INCREMENT	Identificador único da embarcação.
<u>Nome_E</u>	VARCHAR(100)	NOT NULL	Nome de identificação (ex: “Estrela do Norte”).
<u>Capacidade_Max</u>	INT	NOT NULL	Lotação máxima para fins de segurança.
<u>ID_TE</u>	INT	FK, NOT NULL	Referência à categoria (barco, lancha).

Tabela 10 – **Rota**

Campo	Tipo de Dado	Restrição	Descrição de Negócio
<u>ID_Rota</u>	INT	PK, AUTO_INCREMENT	Identificador único do trajeto.
<u>P_Origem</u>	VARCHAR(100)	NOT NULL	Porto de partida.
<u>P_Destino</u>	VARCHAR(100)	NOT NULL	Porto de chegada.
<u>Valor_Base</u>	DECIMAL(10,2)	NOT NULL	Preço padrão do trecho sem taxas adicionadas.

Tabela 11 – Viagem

Campo	Tipo de Dado	Restrição	Descrição de Negócio
<u>ID_Viagem</u>	INT	PK, AUTO_INCREMENT	Identificador do evento de transporte.
Data_Saida	DATE	NOT NULL	Data programada para o início do trajeto.
Hora_Saida	TIME	NOT NULL	Horário exato de partida do porto.
<u>ID_Embarcacao</u>	INT	FK, NOT NULL	Embarcação alocada para o evento.
<u>ID_Rota</u>	INT	FK, NOT NULL	Rota definida para esta viagem específica.

Tabela 12 – Reserva

Campo	Tipo de Dado	Restrição	Descrição de Negócio
<u>ID_Reserva</u>	INT	PK, AUTO_INCREMENT	Identificador do bilhete emitido.
Data_Reserva	DATETIME	DEFAULT CURRENT_TIMESTAMP	Data/hora exata em que a reserva foi gerada.
Status_Reserva	VARCHAR(50)	NOT NULL	Estado (Paga, Pendente, Cancelada).
<u>ID_Viagem</u>	INT	FK, NOT NULL	Viagem correspondente ao bilhete.
<u>ID_Passageiro</u>	INT	FK, NOT NULL	Passageiro que ocupa o assento.

Tabela 13 – Tipo_Embarcacao

Campo	Tipo de Dado	Restrição	Descrição de Negócio
<u>ID_TE</u>	INT	PK, AUTO_INCREMENT	Identificador único da categoria.
<u>Descricao_TE</u>	VARCHAR(100)	NOT NULL	Tipo de casco ou serviço (ex: Balsa, Catamarã).

8.4 Análise de Decisões Técnicas

O uso de `VARCHAR(14)` para o CPF foi planejado para suportar os 11 dígitos numéricos somados aos 3 caracteres de formatação (pontos e traço), permitindo que a aplicação salve o dado já formatado ou limpo sem perda de integridade. A restrição `UNIQUE` impede duplicidades de cadastro na origem. O uso de `DEFAULT CURRENT_TIMESTAMP` em `Reserva` automatiza a auditoria temporal, removendo a responsabilidade da aplicação de gerar marcas de tempo e garantindo que o banco de dados registre o momento real da inserção.

8.5 Hierarquia e Integridade Referencial

As chaves estrangeiras (*FOREIGN KEYS*) funcionam como as guardiãs da integridade referencial, impedindo a existência de “dados órfãos” que causariam falhas sistêmicas. O banco de dados foi projetado para seguir um fluxo de dependência lógico:

1. Uma `Viagem` só pode ser instanciada se houver uma `Rota` e uma `Embarcacao` válidas;
2. Uma `Reserva` serve como o nexos final, vinculando obrigatoriamente um `Passageiro` a uma `Viagem` existente.

Essa hierarquia garante que o banco de dados seja a primeira e mais robusta camada de validação de regras de negócio. Por exemplo, a tentativa de registrar uma embarcação sem um tipo previamente definido será bloqueada pelo motor `InnoDB`, assegurando que inconsistências nunca cheguem ao disco.

8.6 Implementação de Regras de Negócio e Restrições Físicas

A regra mais crítica do sistema Porto Certo é o controle da capacidade máxima das embarcações. Embora a estrutura de tabelas permita o armazenamento, a validação lógica deve ocorrer na camada de banco de dados através de um **Gatilho** (*Trigger*).

Esta abordagem é superior à validação apenas no código da aplicação pois resolve o problema da **concorrência**. Em um cenário onde dois passageiros tentam reservar o último assento exatamente ao mesmo milésimo de segundo, o banco de dados garante a **atomicidade** da operação: o Gatilho (*Trigger*) realiza a contagem de reservas atuais e bloqueia a transação excedente, fornecendo um “Não” definitivo e impedindo o *overbooking*. Além disso:

- **Integridade de Identidade:** O UNIQUE no CPF assegura a unicidade do registro de passageiro.
- **Motor de Estado:** O campo `Status_Reserva` funciona como o motor de transição da reserva, permitindo que processos de *backend* (como cancelamentos por falta de pagamento) operem de forma consistente.

Ao consolidar estas regras na camada física, protegemos o sistema contra “dados sujos” (*dirty data*) provenientes de edições manuais ou múltiplas versões de APIs, garantindo que a arquitetura Porto Certo esteja pronta para a produção com máxima confiabilidade.